

Assessment 2

presentation

MEDA
Reconstructing Missing
Atmospheric Pressure
Measurements



Date: 6th May 2026



Deliverables

assessment 2

Iterative model
development log
(experiments,
hyperparameters,
scores).



Current leaderboard
standing screenshot
with date/time stamp



Documentation of
techniques tried and
insights gained.



XGBoost (eXtreme Gradient Boosting)

1. Regularization

- Adds penalties for complex trees.
- Keeps the model generalizable.

2. Parallel Processing

- Unlike traditional boosting, parts of XGBoost can run in parallel.

3. Handling Missing Values

- Automatically learns the best way to handle missing data.

4. Tree Pruning

- Uses a max depth + pruning strategy to avoid overly complex trees.

5. Weighted Learning

- Can assign importance to different data points.

Experiment 1



Hyperparameter

```
24  model = xgb.XGBRegressor(  
25      n_estimators=100,  
26      learning_rate=0.1,  
27      max_depth=5,  
28      random_state=42,  
29      eval_metric='rmse',  
30      device='cuda'
```



Score

MSE = 8803.98

Experiment 2



Hyperparameters

```
53 model = xgb.XGBRegressor(  
54     n_estimators=200,  
55     learning_rate=0.1,  
56     max_depth=6,  
57     random_state=42,  
58     eval_metric='rmse',  
59     device='cuda'  
60 )
```

For experiment 2, we included cyclic encoding to handle temporal features.



Score

MSE = 8791.79

1st experiment vs 2nd experiment

```
mars-environmental-dynamics-analyzer-meda-virtual-sensor-recovery > firstattemptxgb.csv
```

1	row_id,PRESSURE
2	684737856,760.0496
3	684737857,759.9401
4	684737858,760.0496
5	684737859,760.0496
6	684737860,760.0496
7	684737861,760.0496
8	684737862,760.0496
9	684737863,760.0496
10	684737864,760.89746
11	684737865,760.89746
12	684737866,761.007
13	684737867,760.89746
14	684737868,760.89746
15	684737869,760.89746
16	684737870,761.007
17	684737871,761.007
18	684737872,760.89746
19	684737873,760.89746
20	684737874,760.89746
21	684737875,761.007
22	684737876,761.007

```
mars-environmental-dynamics-analyzer-meda-virtual-sensor-recovery > v2_submission_time_fe
```

1	row_id,PRESSURE
2	684737856,759.3479
3	684737857,759.4248
4	684737858,759.4248
5	684737859,759.4248
6	684737860,759.4248
7	684737861,759.4248
8	684737862,759.4248
9	684737863,759.4248
10	684737864,760.8727
11	684737865,760.8727
12	684737866,760.8727
13	684737867,760.8727
14	684737868,760.8727
15	684737869,760.8727
16	684737870,760.8727
17	684737871,760.8727
18	684737872,760.8727
19	684737873,760.8727
20	684737874,760.72406
21	684737875,760.72406
22	684737876,760.72406

You can see that there are some changes to the predicted pressures value at the same row_id

During the 1st
attempt, our
approach is..

1st experiment vs 2nd experiment

To move fast, we kept our data preparation extremely simple:

- Kept it to the Numbers: We only fed the model raw numerical data (like temperatures and angles). We temporarily threw out complex text data, like the Martian timestamps (LMST and LTST), to save time.
- Martian sensor data is messy and full of blank spots (NaNs). Instead of trying to manually guess and fill in those blanks, we used an algorithm called XGBoost. XGBoost is smart enough to handle missing data automatically without breaking.
- We turned on GPU acceleration (using a graphics card instead of a standard processor) so the model could learn from millions of rows in just a few seconds.

1st experiment vs 2nd experiment

```
# Baseline Model:
model = xgb.XGBRegressor(
    n_estimators=100, # Fewer trees
    learning_rate=0.1,
    max_depth=5,      # Shallower trees
    ...
)

# V2 Model (Iterated):
model = xgb.XGBRegressor(
    n_estimators=200, # Increased!
    learning_rate=0.1,
    max_depth=6,      # Increased!
    ...
)
```

Tuning the XGBoost Hyperparameters

Where it happened:

Inside the `xgb.XGBRegressor()` initialization.

The Iteration:

Because we added high-quality temporal features, the model needed a bit more depth and time to discover the complex relationships between the time of day, the solar angles, and the pressure.

The Code Changes:

`n_estimators` (Number of Trees): Increased from 100 to 200. This gives the model 100 extra iterations to learn from its mistakes and fine-tune its predictions based on the new time features.

`max_depth` (Tree Complexity): Increased from 5 to 6. A depth of 6 allows the decision trees to ask one more "layer" of questions (e.g., "If the time is 14:00 AND the solar zenith angle is X AND the humidity temperature is Y...").

1st experiment vs 2nd experiment

```
def extract_cyclic_time(df, time_col='LMST'):
    """
    Extracts time from the Martian timestamp and applies cyclic encoding.
    Works for LMST (e.g., '00001M16:05:31.315') or LTST.
    """
    print(f"    -> Extracting features from {time_col}...")

    # 1. Extract the HH:MM:SS portion using regular expressions
    time_strings = df[time_col].str.extract(r'(\d{2}:\d{2}:\d{2})')[0]

    # 2. Parse into datetime objects
    parsed_times = pd.to_datetime(time_strings, format='%H:%M:%S', errors='coerce')

    # 3. Convert to a continuous decimal hour (e.g., 16:30 becomes 16.5)
    hour_of_sol = parsed_times.dt.hour + (parsed_times.dt.minute / 60.0) + (parsed_times.dt.second / 3600.0)

    # 4. Cyclic Encoding (Sine and Cosine transforms)
    # 24.0 represents the full Martian day (Sol)
    df['Time_Sin'] = np.sin(2 * np.pi * hour_of_sol / 24.0)
    df['Time_Cos'] = np.cos(2 * np.pi * hour_of_sol / 24.0)

    return df
```

The core difference is that the baseline completely threw away the time data, while the V2 pipeline actively translates that time into a mathematical language the model can understand, and then scales up the model's brainpower to use it

Leaderboard

+

Search

Mars Environmental Dynamics Analyzer (MEDA)

Overview

Data

Code

Models

Discussion

Leaderboard

Rules

Team

Submissions

Public

Private

Submit Prediction

This leaderboard is calculated with approximately 50% of the test data. The final results will be based on the other 50%, so the final standings may be different.

#	Team	Members	Score	Entries	Last	Join
1	Himanshu Dhiman		8313.049752723509	1	4mo	
2	Mubeena Hussain		8759.963132407929	1	2mo	
3	Penunggang Kaggle	  	8791.793925923030	2	10h	
 Your Best Entry! Your most recent submission scored 8791.793925923030, which is an improvement over your previous score of 8803.984238502480. Great job! Tweet this						
	DL_Pressure_map_SAMPLE.csv		31830.593329302501	1	4mo	
4	sponish_flea		31830.593329302501	1	3mo	
	sample_submission.csv		3799158591104.9301...	1	4mo	

Mars Environmental Dy

ENG

100%

12:57 PM

6/5/2026



Future experiment

```
... --- Starting Experiment: LSTM_Exp_01_Baseline ---
Loading full datasets...
Calculating missing data across the entire 8 million rows...
Dynamically dropping these sensors (>50% missing): ['MASTHEAD_AZIMUTH', 'MASTHEAD_ELEVATION', 'PRESSURE_UNCERTAINTY', 'LOCAL_RELATIVE_HUMIDITY', 'LOCAL_RELATIVE_HUMIDITY_UNCERTAINTY', 'VOLUME_MIXING_RATIO', 'VOLUME_MIXING_RATIO_UNCERTAINTY']
Interpolating missing values...
Scaling data...
Setting up memory-efficient data pipelines...
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer.
  super().__init__(**kwargs)
Starting training...
Epoch 1/20
6352/6352 ————— 0s 136ms/step - loss: 468372.1734WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using the JSON format instead.
6352/6352 ————— 1064s 167ms/step - loss: 409479.4062 - val_loss: 300840.6250
Epoch 2/20
6352/6352 ————— 0s 130ms/step - loss: 234992.3460WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using the JSON format instead.
6352/6352 ————— 1022s 161ms/step - loss: 196277.0000 - val_loss: 124716.2109
Epoch 3/20
6352/6352 ————— 0s 134ms/step - loss: 86665.3020WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using the JSON format instead.
6352/6352 ————— 1045s 164ms/step - loss: 65861.6484 - val_loss: 28109.3340
Epoch 4/20
6352/6352 ————— 0s 132ms/step - loss: 16113.4265WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using the JSON format instead.
6352/6352 ————— 1035s 163ms/step - loss: 10883.7822 - val_loss: 587.5867
Epoch 5/20
6352/6352 ————— 0s 130ms/step - loss: 4066.8150WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using the JSON format instead.
6352/6352 ————— 1014s 160ms/step - loss: 4106.1055 - val_loss: 84.1390
Epoch 6/20
6352/6352 ————— 0s 130ms/step - loss: 4093.9850WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using the JSON format instead.
6352/6352 ————— 1021s 161ms/step - loss: 4098.1533 - val_loss: 74.7997
Epoch 7/20
6352/6352 ————— 1022s 161ms/step - loss: 4003.0798 - val_loss: 80.7020
Epoch 8/20
6352/6352 ————— 0s 130ms/step - loss: 3923.9221WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using the JSON format instead.
6352/6352 ————— 1020s 161ms/step - loss: 3943.5093 - val_loss: 58.4629
Epoch 9/20
6352/6352 ————— 0s 126ms/step - loss: 3851.4452WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using the JSON format instead.
6352/6352 ————— 996s 157ms/step - loss: 3882.8137 - val_loss: 55.9327
Epoch 10/20
6352/6352 ————— 994s 156ms/step - loss: 3825.6248 - val_loss: 71.5608
Epoch 11/20
129/6352 ————— 14:01 135ms/step - loss: 3853.1165
```




Penunggang Kaggle

Thank You